

GALE-SHAPLEY (*men, women, rankings*)

```

1  assign each woman and man as free
2  while some woman  $w$  is free
3      let  $m$  be the first man on  $w$ 's ranked list to whom she has not proposed
4      if  $m$  is free
5           $w$  and  $m$  become engaged to each other (and not free)
6      elseif  $m$  ranks  $w$  higher than the woman  $w'$  he is currently engaged to
7           $m$  breaks the engagement to  $w'$ , who becomes free
8           $w$  and  $m$  become engaged to each other (and not free)
9      else  $m$  rejects  $w$ , with  $w$  remaining free
10 return the stable matching consisting of the engaged pairs

```

2. Emma proposes to Davis. Davis is free, so that Emma and Davis become engaged and no longer free.
3. Lacey proposes to Brent. Brent is engaged to Wanda, but he prefers Lacey. Brent breaks the engagement to Wanda, who becomes free. Lacey and Brent become engaged, with Lacey no longer free.
4. Karen proposes to Brent. Brent is engaged to Lacey, whom he prefers to Karen. Brent rejects Karen, who remains free.
5. Karen proposes to Hank. Hank is free, so that Karen and Hank become engaged and no longer free.
6. Wanda proposes to Hank. Hank is engaged to Karen, but he prefers Wanda. Hank breaks the engagement with Karen, who becomes free. Wanda and Hank become engaged, with Wanda no longer free.
7. Karen proposes to Davis. Davis is engaged to Emma, but he prefers Karen. Davis breaks the engagement to Emma, who becomes free. Karen and Davis become engaged, with Karen no longer free.
8. Emma proposes to Hank. Hank is engaged to Wanda, whom he prefers to Emma. Hank rejects Emma, who remains free.
9. Emma proposes to Oscar. Oscar is free, so that Emma and Oscar become engaged and no longer free.

At this point, everyone is engaged and nobody is free, so the **while** loop terminates. The procedure returns the stable matching we saw earlier.

The following theorem shows that not only does GALE-SHAPLEY terminate, but that it always returns a stable matching, thereby proving that a stable matching always exists.

Theorem 25.9

The procedure GALE-SHAPLEY always terminates and returns a stable matching.

Proof Let's first show that the **while** loop of lines 2–9 always terminates, so that the procedure terminates. The proof is by contradiction. If the loop fails to terminate, it is because some woman remains free. In order for a woman to remain free, she must have proposed to all the men and been rejected by each one. In order for a man to reject a woman, he must be already engaged. Therefore, all the men are engaged. Once engaged, a man stays engaged (though not necessarily to the same woman). There are an equal number n of women and men, however, which means that every woman is engaged, leading to the contradiction that no women are free. We must also show that the **while** loop makes a bounded number of iterations. Since each of the n women goes through her ranking of the n men in order, possibly not reaching the end of her list, the total number of iterations is at most n^2 . Therefore, the **while** loop always terminates, and the procedure returns a matching.

We need to show that there are no blocking pairs. We first observe that once a man m is engaged to a woman w , all subsequent actions for m occur in lines 6–8. Therefore, once a man is engaged, he stays engaged, and any time he breaks an engagement to a woman w , it's for a woman whom he prefers to w . Suppose that a woman w is matched with a man m , but she prefers man m' . We'll show that w and m' is not a blocking pair, because m' does not prefer w to his partner. Because w ranks m' higher than m , she must have proposed to m' before proposing to m , and m' either rejected her proposal or accepted it and later broke the engagement. If m' rejected the proposal from w , it is because he was already engaged to some woman he prefers to w . If m' accepted and later broke the engagement, he was at some point engaged to w but later accepted a proposal from a woman he prefers to w . In either case, he ultimately ends up with a partner whom he prefers to w . We conclude that even though w might prefer m' to her partner m , it is not also the case that m' prefers w to his partner. Therefore, the procedure returns a matching containing no blocking pairs. ■

Exercise 25.2-1 asks you to provide the proof of the following corollary.

Corollary 25.10

Given preference rankings for n women and n men, the Gale-Shapley algorithm can be implemented to run in $O(n^2)$ time. ■

Because line 2 can choose any free woman, you might wonder whether different choices can produce different stable matchings. The answer is no: as the following

theorem shows, every execution of the GALE-SHAPLEY produces exactly the same result. Moreover, the stable matching returned is optimal for the women.

Theorem 25.11

Regardless of how women are chosen in line 2 of GALE-SHAPLEY, the procedure always returns the same stable matching, and in this stable matching, each woman has the best partner possible in any stable matching.

Proof The proof that each woman has the best partner possible in any stable matching is by contradiction. Suppose that the GALE-SHAPLEY procedure returns a stable matching M , but that there is a different stable matching M' in which some woman w prefers her partner m' to the partner m she has in M . Because w ranks m' higher than m , she must have proposed to m' before proposing to m . Then there is a woman w' whom m' prefers to w , and m' was already engaged to w' when w proposed or m' accepted the proposal from w and later broke the engagement in favor of w' . Either way, there is a moment when m' decided against w in favor of w' . Now suppose, without loss of generality, that this moment was the first time that any man rejected a partner who belongs to some stable matching.

We claim that w' cannot have a partner m'' in a stable matching whom she prefers to m' . If there were such a man m'' , then in order for w' to propose to m' , she would have proposed to m'' and been rejected at some point before proposing to m' . If m' accepted the proposal from w and later broke it to accept w' , then since this was the first rejection in a stable matching, we get the contradiction that m'' could not have rejected w' beforehand. If m' was already engaged to w' when w proposed, then again, m'' could not have rejected w' beforehand, thus proving the claim.

Since w' does not prefer anyone to m' in a stable matching and w' is not matched with m' in M' (because m' is matched with w in M'), w' prefers m' to her partner in M' . Since w' prefers m' over her partner in M' and m' prefers w' over his partner w in M' , the pair w' and m' is a blocking pair in M' . Because M' has a blocking pair, it cannot be a stable matching, thereby contradicting the assumption that there exists some stable matching in which each woman has the best partner possible other than the matching M returned by GALE-SHAPLEY.

We put no condition on the execution of the procedure, which means that all possible orders in which line 2 selects women result in the same stable matching being returned. ■

Corollary 25.12

There can be stable matchings that the GALE-SHAPLEY procedure does not return.

Proof Theorem 25.11 says that for a given set of rankings, GALE-SHAPLEY returns just one matching, no matter how it chooses women in line 2. The earlier ex-

ample of three women and three men with three different stable matchings shows that there can be multiple stable matchings for a given set of rankings. A call of GALE-SHAPLEY is capable of returning only one of these stable matchings. ■

Although the GALE-SHAPLEY procedure gives the best possible outcome for the women, the following corollary shows that it also produces the worst possible outcome for the men.

Corollary 25.13

In the stable matching returned by the procedure GALE-SHAPLEY, each man has the worst partner possible in any stable matching.

Proof Let M be the matching returned by a call to GALE-SHAPLEY. Suppose that there is another stable matching M' and a man m who prefers his partner w in M to his partner w' in M' . Let the partner of w in M' be m' . By Theorem 25.11, m is the best partner that w can have in any stable matching, which means that w prefers m to m' . Since m prefers w to w' , the pair w and m is a blocking pair in M' , contradicting the assumption that M' is a stable matching. ■

Exercises

25.2-1

Describe how to implement the Gale-Shapley algorithm so that it runs in $O(n^2)$ time.

25.2-2

Is it possible to have an unstable matching with just two women and two men? If so, provide and justify an example. If not, argue why not.

25.2-3

The National Resident Matching Program differs from the scenario for the stable-marriage problem set out in this section in two ways. First, a hospital may be matched with more than one student, so that hospital h takes $r_h \geq 1$ students. Second, the number of students might not equal the number of hospitals. Describe how to modify the Gale-Shapley algorithm to fit the requirements of the National Resident Matching Program.

25.2-4

Prove the following property, which is known as *weak Pareto optimality*:

Let M be the stable matching produced by the GALE-SHAPLEY procedure, with women proposing to men. Then, for a given instance of the stable-marriage problem there is no matching—stable or unstable—such that every

woman has a partner whom she prefers to her partner in the stable matching M .

25.2-5

The *stable-roommates problem* is similar to the stable-marriage problem, except that the graph is a complete graph, not bipartite, with an even number of vertices. Each vertex represents a person, and each person ranks all the other people. The definitions of blocking pairs and stable matching extend in the natural way: a blocking pair comprises two people who both prefer each other to their current partner, and a matching is stable if there are no blocking pairs. For example, consider four people—Wendy, Xenia, Yolanda, and Zelda—with the following preference lists:

Wendy: Xenia, Yolanda, Zelda
 Xenia: Wendy, Zelda, Yolanda
 Yolanda: Wendy, Zelda, Xenia
 Zelda: Xenia, Yolanda, Wendy

You can verify that the following matching is stable:

Wendy and Xenia
 Yolanda and Zelda

Unlike the stable-marriage problem, the stable-roommates problem can have inputs for which no stable matching exists. Find such an input and explain why no stable matching exists.

25.3 The Hungarian algorithm for the assignment problem

Let us once again add some information to a complete bipartite graph $G = (V, E)$, where $V = L \cup R$. This time, instead of having the vertices of each side rank the vertices on the other side, we assign a weight to each edge. Again, let's assume that the vertex sets L and R each contain n vertices, so that the graph contains n^2 edges. For $l \in L$ and $r \in R$, denote the weight of edge (l, r) by $w(l, r)$, which represents the utility gained by matching vertex l with vertex r .

The goal is to find a perfect matching M^* (see Exercises 25.1-5 and 25.1-6) whose edges have the maximum total weight over all perfect matchings. That is, letting $w(M) = \sum_{(l,r) \in M} w(l, r)$ denote the total weight of the edges in matching M , we want to find a perfect matching M^* such that

$$w(M^*) = \max \{w(M) : M \text{ is a perfect matching}\} .$$

We call finding such a maximum-weight perfect matching the *assignment problem*. A solution to the assignment problem is a perfect matching that maximizes the total utility. Like the stable-marriage problem, the assignment problem finds a matching that is “good,” but with a different definition of good: maximizing total value rather than achieving stability.

Although you could enumerate all $n!$ perfect matchings to solve the assignment problem, an algorithm known as the *Hungarian algorithm* solves it much faster. This section will prove an $O(n^4)$ time bound, and Problem 25-2 asks you to refine the algorithm to reduce the running time to $O(n^3)$. Instead of working with the complete bipartite graph G , the Hungarian algorithm works with a subgraph of G called the “equality subgraph.” The equality subgraph, which is defined below, changes over time and has the beneficial property that any perfect matching in the equality subgraph is also an optimal solution to the assignment problem.

The equality subgraph depends on assigning an attribute h to each vertex. We call h the *label* of a vertex, and we say that h is a *feasible vertex labeling* of G if

$$l.h + r.h \geq w(l, r) \text{ for all } l \in L \text{ and } r \in R .$$

A feasible vertex labeling always exists, such as the *default vertex labeling* given by

$$l.h = \max \{w(l, r) : r \in R\} \quad \text{for all } l \in L , \quad (25.1)$$

$$r.h = 0 \quad \text{for all } r \in R . \quad (25.2)$$

Given a feasible vertex labeling h , the *equality subgraph* $G_h = (V, E_h)$ of G consists of the same vertices as G and the subset of edges

$$E_h = \{(l, r) \in E : l.h + r.h = w(l, r)\} .$$

The following theorem ties together a perfect matching in an equality subgraph and an optimal solution to the assignment problem.

Theorem 25.14

Let $G = (V, E)$, where $V = L \cup R$, be a complete bipartite graph where each edge $(l, r) \in E$ has weight $w(l, r)$. Let h be a feasible vertex labeling of G and G_h be the equality subgraph of G . If G_h contains a perfect matching M^* , then M^* is an optimal solution to the assignment problem on G .

Proof If G_h contains a perfect matching M^* , then because G_h and G have the same sets of vertices, M^* is also a perfect matching in G . Because each edge of M^* belongs to G_h and each vertex has exactly one incident edge from any perfect matching, we have

$$\begin{aligned}
w(M^*) &= \sum_{(l,r) \in M^*} w(l,r) \\
&= \sum_{(l,r) \in M^*} (l.h + r.h) \quad (\text{because all edges in } M^* \text{ belong to } G_h) \\
&= \sum_{l \in L} l.h + \sum_{r \in R} r.h \quad (\text{because } M^* \text{ is a perfect matching}) .
\end{aligned}$$

Letting M be any perfect matching in G , we have

$$\begin{aligned}
w(M) &= \sum_{(l,r) \in M} w(l,r) \\
&\leq \sum_{(l,r) \in M} (l.h + r.h) \quad (\text{because } h \text{ is a feasible vertex labeling}) \\
&= \sum_{l \in L} l.h + \sum_{r \in R} r.h \quad (\text{because } M \text{ is a perfect matching}) .
\end{aligned}$$

Thus, we have

$$w(M) \leq \sum_{l \in L} l.h + \sum_{r \in R} r.h = w(M^*) , \quad (25.3)$$

so that M^* is a maximum-weight perfect matching in G . ■

The goal now becomes finding a perfect matching in an equality subgraph. Which equality subgraph? It does not matter! We have free rein to not only choose an equality subgraph, but to change which equality subgraph we choose as we go along. We just need to find *some* perfect matching in *some* equality subgraph.

To understand the equality subgraph better, consider again the proof of Theorem 25.14 and, in the second half, let M be any matching. The proof is still valid, in particular, inequality (25.3): the weight of any matching is always at most the sum of the vertex labels. If we choose any set of vertex labels that define an equality subgraph, then a maximum-cardinality matching in this equality subgraph has total value at most the sum of the vertex labels. If the set of vertex labels is the “right” one, then it will have total value equal to $w(M^*)$, and a maximum-cardinality matching in the equality subgraph is also a maximum-weight perfect matching. The Hungarian algorithm repeatedly modifies the matching and the vertex labels in order to achieve this goal.

The Hungarian algorithm starts with any feasible vertex labeling h and any matching M in the equality subgraph G_h . It repeatedly finds an M -augmenting path P in G_h and, using Lemma 25.1, updates the matching to be $M \oplus P$, thereby incrementing the size of the matching. As long as there is some equality subgraph that contains an M -augmenting path, the size of the matching can increase, until a perfect matching is achieved.

Four questions arise:

1. What initial feasible vertex labeling should the algorithm start with? Answer: the default vertex labeling given by equations (25.1) and (25.2).
2. What initial matching in G_h should the algorithm start with? Short answer: any matching, even an empty matching, but a greedy maximal matching works well.
3. If an M -augmenting path exists in G_h , how to find it? Short answer: use a variant of breadth-first search similar to the second phase of the procedure used in the Hopcroft-Karp algorithm to find a maximal set of shortest M -augmenting paths.
4. What if the search for an M -augmenting path fails? Short answer: update the feasible vertex labeling to bring in at least one new edge.

We'll elaborate on the short answers using the example that starts in Figure 25.4. Here, $L = \{l_1, l_2, \dots, l_7\}$ and $R = \{r_1, r_2, \dots, r_7\}$. The edge weights appear in the matrix shown in part (a), where the weight $w(l_i, r_j)$ appears in row i and column j . The feasible vertex labels, given by the default vertex labeling, appear to the left of and above the matrix. Matrix entries in red indicate edges (l_i, r_j) for which $l_i.h + r_j.h = w(l_i, r_j)$, that is, edges in the equality subgraph G_h appearing in part (b) of the figure.

Greedy maximal bipartite matching

There are several ways to implement a greedy method to find a maximal bipartite matching. The procedure GREEDY-BIPARTITE-MATCHING shows one. Edges in Figure 25.4(b) highlighted in blue indicate the initial greedy maximal matching in G_h . Exercise 25.3-2 asks you to show that the GREEDY-BIPARTITE-MATCHING procedure returns a matching that is at least half the size of a maximum matching.

GREEDY-BIPARTITE-MATCHING(G)

```

1   $M = \emptyset$ 
2  for each vertex  $l \in L$ 
3      if  $l$  has an unmatched neighbor in  $R$ 
4          choose any such unmatched neighbor  $r \in R$ 
5           $M = M \cup \{(l, r)\}$ 
6  return  $M$ 
```

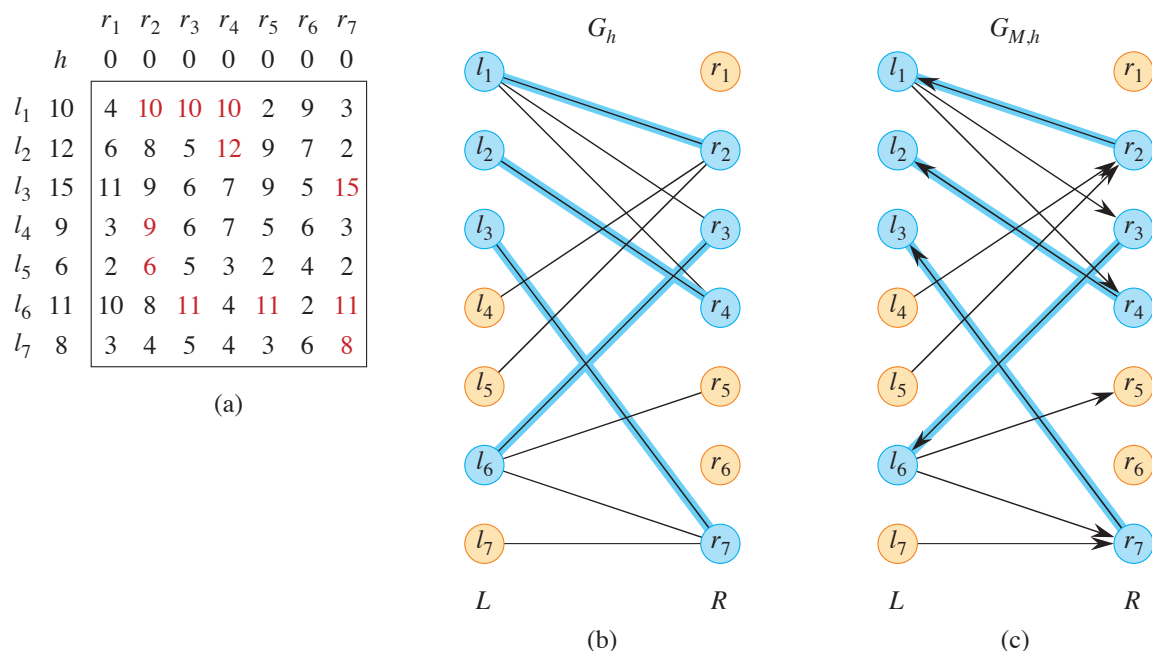



Figure 25.4 The start of the Hungarian algorithm. (a) The matrix of edge weights for a bipartite graph with $L = \{l_1, l_2, \dots, l_7\}$ and $R = \{r_1, r_2, \dots, r_7\}$. The value in row i and column j indicates $w(l_i, r_j)$. Feasible vertex labels appear above and next to the matrix. Red entries correspond to edges in the equality subgraph. (b) The equality subgraph G_h . Edges highlighted in blue belong to the initial greedy maximal matching M . Blue vertices are matched, and tan vertices are unmatched. (c) The directed equality subgraph $G_{M,h}$ created from G_h by directing edges in M from R to L and all other edges from L to R .

Finding an M -augmenting path in G_h

To find an M -augmenting path in the equality subgraph G_h with a matching M , the Hungarian algorithm first creates the **directed equality subgraph** $G_{M,h}$ from G_h , just as the Hopcroft-Karp algorithm creates G_M from G . As in the Hopcroft-Karp algorithm, you can think of an M -augmenting path as starting from an unmatched vertex in L , ending at an unmatched vertex in R , taking unmatched edges from L to R , and taking matched edges from R to L . Thus, $G_{M,h} = (V, E_{M,h})$, where

$$E_{M,h} = \{(l, r) : l \in L, r \in R, \text{ and } (l, r) \in E_h - M\} \quad (\text{edges from } L \text{ to } R) \\ \cup \{(r, l) : r \in R, l \in L, \text{ and } (l, r) \in M\} \quad (\text{edges from } R \text{ to } L).$$

Because an M -augmenting path in the directed equality subgraph $G_{M,h}$ is also an M -augmenting path in the equality subgraph G_h , it suffices to find M -augmenting paths in $G_{M,h}$. Figure 25.4(c) shows the directed equality subgraph $G_{M,h}$ corresponding to the equality subgraph G_h and matching M from part (b) of the figure.

With the directed equality subgraph $G_{M,h}$ in hand, the Hungarian algorithm searches for an M -augmenting path from any unmatched vertex in L to any unmatched vertex in R . Any exhaustive graph-search method suffices. Here, we'll use breadth-first search, starting from all the unmatched vertices in L (just as the Hopcroft-Karp algorithm does when creating the dag H), but stopping upon first discovering some unmatched vertex in R . Figure 25.5 shows the idea. To start from all the unmatched vertices in L , initialize the first-in, first-out queue with all the unmatched vertices in L , rather than just one source vertex. Unlike the dag H in the Hopcroft-Karp algorithm, here each vertex needs just one predecessor, so that the breadth-first search creates a *breadth-first forest* $F = (V_F, E_F)$. Each unmatched vertex in L is a root in F .

In Figure 25.5(g), the breadth-first search has found the M -augmenting path $\langle (l_4, r_2), (r_2, l_1), (l_1, r_3), (r_3, l_6), (l_6, r_5) \rangle$. Figure 25.6(a) shows the new matching created by taking the symmetric difference of the matching M in Figure 25.5(a) with this M -augmenting path.

When the search for an M -augmenting path fails

Having updated the matching M from an M -augmenting path, the Hungarian algorithm updates the directed equality subgraph $G_{M,h}$ according to the new matching and then starts a new breadth-first search from all the unmatched vertices in L . Figure 25.6 shows the start of this process, picking up from Figure 25.5.

In Figure 25.6(d), the queue contains vertices l_4 and l_3 . Neither of these vertices has an edge that leaves it, however, so that once these vertices are removed from the queue, the queue becomes empty. The search terminates at this point, before discovering an unmatched vertex in R to yield an M -augmenting path. Whenever this situation occurs, the most recently discovered vertices must belong to L . Why? Whenever an unmatched vertex in R is discovered, the search has found an M -augmenting path, and when a matched vertex in R is discovered, it has an unvisited neighbor in L , which the search can then discover.

Recall that we have the freedom to work with any equality subgraph. We can change the directed equality subgraph “on the fly,” as long as we do not counteract the work already done. The Hungarian algorithm updates the feasible vertex labeling h to fulfill the following criteria:

1. No edge in the breadth-first forest F leaves the directed equality subgraph.
2. No edge in the matching M leaves the directed equality subgraph.
3. At least one edge (l, r) , where $l \in L \cap V_F$ and $r \in R - V_F$ goes into E_h , and hence into $E_{M,h}$. Therefore, at least one vertex in R will be newly discovered.

Thus, at least one new edge enters the directed equality subgraph, and any edge that leaves the directed equality subgraph belongs to neither the matching M nor